



UNIVERSIDADE PAULISTA – UNIP

**CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS**

PROJETO INTEGRADO MULTIDISCIPLINAR III

Willian Augusto Quincas Senna

RA: H78BIA3

Diego Santos Rodrigues

RA: R862JH7

Cauã Brandão Da Costa

RA: R8327B0

Luiz Gustavo Da Silva Borges

RA: H4062J7

**DESENVOLVIMENTO DE PLATAFORMA WEB DE AVALIAÇÃO E APOIO À
APRENDIZAGEM PARA UMA EMPRESA FICTÍCIA DE TECNOLOGIA
EDUCACIONAL**

São José dos Campos – SP

2026

Willian Augusto Quincas Senna

RA: H78BIA3

Diego Santos Rodrigues

RA: R862JH7

Cauã Brandão Da Costa

RA: R8327B0

Luiz Gustavo Da Silva Borges

RA: H4062J7

**DESENVOLVIMENTO DE PLATAFORMA WEB DE AVALIAÇÃO E APOIO À
APRENDIZAGEM PARA UMA EMPRESA FICTÍCIA DE TECNOLOGIA
EDUCACIONAL**

Projeto Integrado Multidisciplinar em Análise e
Desenvolvimento de Sistemas

Projeto Integrado Multidisciplinar para obtenção do
título de Tecnólogo em Análise e Desenvolvimento
de Sistemas, apresentado à Universidade Paulista
– UNIP.

Orientador(a): Prof^ª. Roberto Cordeiro Waltz

São José dos Campos – SP

2026

RESUMO

O presente trabalho, desenvolvido no âmbito do Projeto Integrado Multidisciplinar III (PIM III) do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Paulista (UNIP), apresenta o projeto e o desenvolvimento de uma plataforma web de avaliação e apoio à aprendizagem denominada Tech Quest. A empresa fictícia criada para contextualizar o projeto atua no segmento de tecnologia educacional e tem como público-alvo estudantes de cursos de tecnologia, profissionais em transição de carreira e tutores especialistas na área de programação. O problema identificado consiste na dificuldade de acompanhar o desempenho individual dos aprendizes de forma estruturada, centralizada e acessível em diferentes dispositivos.

O objetivo do trabalho foi desenvolver o projeto da plataforma integrando as disciplinas do semestre, com ênfase em engenharia de software ágil, modelagem de dados, programação orientada a objetos em C#, desenvolvimento web responsivo, UX/UI design e análise de dados. Para o desenvolvimento, adotou-se uma abordagem baseada em práticas ágeis, com levantamento de requisitos, definição de backlog e organização por iterações. Foram identificadas três personas principais: o estudante Felipe Almeida, a tutora Ana Souza e o administrador Carlos Mendes, cujas necessidades orientaram as decisões de design e funcionalidade ao longo de todo o projeto.

A plataforma permite que tutores criem cursos compostos por material de estudo e avaliações; que estudantes consumam o conteúdo, realizem provas, acumulem medalhas e emitam certificados de conclusão; e que administradores moderem o conteúdo publicado e acompanhem indicadores gerais do sistema. O protótipo foi implementado com HTML, CSS e JavaScript com dados simulados em arquivos JSON, adotando arquitetura mobile-first e responsividade para diferentes dispositivos e resoluções. Os aspectos de acessibilidade, inclusão e conformidade com a Lei Geral de Proteção de Dados (LGPD) foram considerados na definição dos requisitos do sistema.

Os resultados demonstram a viabilidade técnica da solução proposta e evidenciam a integração entre as competências desenvolvidas ao longo do semestre, contribuindo para a formação prático-teórica dos integrantes do grupo.

Palavras-chave: Plataforma educacional. Tecnologia educacional. Desenvolvimento web responsivo. Engenharia de software ágil. UX/UI design. Programação orientada a objetos em C#.

ABSTRACT

This work, developed within the scope of the Integrated Multidisciplinary Project III (PIM III) of the Bachelor of Technology in Systems Analysis and Development at Universidade Paulista (UNIP), presents the design and development of a web-based assessment and learning support platform called Tech Quest. The fictional company created to contextualize the project operates in the educational technology sector and targets technology students, career-changers, and expert tutors in the field of programming. The identified problem concerns the difficulty of tracking individual learner performance in a structured, centralized, and device-accessible manner.

The objective of this work was to develop the platform project by integrating the semester's disciplines, with emphasis on agile software engineering, data modeling, object-oriented programming in C#, responsive web development, UX/UI design, and data analysis. An agile-based approach was adopted for development, including requirements gathering, backlog definition, and iteration planning. Three main personas were identified — student Felipe Almeida, tutor Ana Souza, and administrator Carlos Mendes — whose needs guided design and functionality decisions throughout the project.

The platform allows tutors to create courses composed of study materials and assessments; students to consume content, take exams, earn badges, and issue completion certificates; and administrators to moderate published content and monitor system-wide indicators. The prototype was implemented using HTML, CSS and JavaScript with data simulated through JSON files, following a mobile-first architecture and full responsiveness across different devices and screen sizes. Accessibility, inclusion, and compliance with Brazil's General Data Protection Law (LGPD) were also considered in the system requirements definition.

The results demonstrate the technical feasibility of the proposed solution and highlight the integration of competencies developed throughout the semester, contributing to the practical and theoretical training of the group members.

Keywords: Educational platform. Educational technology. Responsive web development. Agile software engineering. UX/UI design. Object-oriented programming in C#.

Lista de ilustrações

Figura 1 - Diagrama Casos de Uso.....	28
Figura 2 - Diagrama Caso de Uso Estudante	29
Figura 3 - Diagrama Caso De Uso Usuario.....	29
Figura 4 - Diagrama Caso de Uso Administrador	30
Figura 5 - Diagrama de Entidade e Relacionamento	33
Figura 6 - Diagrama de Classes.....	36

Lista de tabelas

Tabela 1 - Requisitos Funcionais.....	25
Tabela 2 - Requisitos Não Funcionais	26
Tabela 3 - Backlog	26
Tabela 4 - Sprints.....	30
Tabela 5 - Persona 1	39
Tabela 6 - Persona 2.....	40
Tabela 7 - Persona 3.....	41
Tabela 8 - Cores design.....	42
Tabela 9 - Heurísticas	43
Tabela 10 - Contribuição para o sistema	46
Tabela 11 - Resultado Objetivos.....	47

Sumário

1 INTRODUÇÃO	11
2 CONTEXTUALIZAÇÃO TEÓRICA.....	12
2.1 Engenharia de Software Ágil Aplicada.....	12
2.2 Modelagem de Banco de Dados e NoSQL.....	13
2.3 Programação Orientada a Objetos (C#).....	14
2.4 Desenvolvimento Web Responsivo.....	15
2.5 UX e UI Design	16
2.6 Machine Learning e Análise de Dados	17
2.7 Comunicação, Liderança, Negociação	18
2.8 LIBRAS	19
3 DESENVOLVIMENTO	21
3.1 Definição e Caracterização do Negócio Fictício	22
3.1.1 Denominação da empresa.....	22
3.1.2 Segmento de atuação.....	22
3.1.3 Público-alvo	22
3.1.4 Produtos e serviços oferecidos.....	23
3.1.5 Necessidades e problemas que justificam a implantação do sistema.....	24
3.2 Engenharia de Software Ágil Aplicada.....	24
3.2.1 Identificação das partes interessadas (stakeholders).....	24
3.2.2 Requisitos funcionais.....	25
3.2.3 Requisitos não funcionais.....	25
3.2.4 Backlog do produto.....	26
3.2.5 Priorização das funcionalidades	26
3.2.6 Definição de iterações e critérios de aceite.....	27
3.2.7 Diagramas	27
3.2.8 Sprints.....	30
3.3 Modelagem de Banco de Dados e NoSQL.....	31
3.3.1 Definição da Modelagem.....	31
3.3.2 Modelo conceitual de dados.....	33
3.3.3 Justificativa técnica para a adoção de banco de dados relacional e/ou NoSQL.....	34
3.4 Programação Orientada a Objetos (C#).....	35
3.4.1 Arquitetura geral do projeto	35
3.5 Desenvolvimento Web Responsivo	37
3.5.1 Tecnologias utilizadas	37
3.5.2 Arquitetura do sistema.....	38
3.5.3 Responsividade e adaptação de layout.....	38
3.5.4 Principais telas desenvolvidas.....	38
3.6 UX e UI Design	39
3.6.1 Definição de personas.....	39
3.6.2 Fluxos de navegação.....	41
3.6.3 Wireframes e protótipos.....	41
3.6.4 Sistema de design	42
3.6.5 Análise de usabilidade.....	43
3.7 Machine Learning e Análise de Dados	43
3.7.1 Tipos de dados relevantes para análise.....	43
3.7.2 Aplicação de técnicas de análise de dados.....	44
3.7.3 Geração de relatórios e indicadores.....	44
3.7.4 Aplicação de machine learning.....	44

3.7.5 Interpretação dos resultados e recomendações.....	45
3.8 Comunicação, Liderança, Negociação e LIBRAS.....	45
3.8.1 Estratégias de comunicação.....	45
3.8.2 Situações simuladas de liderança e negociação.....	45
3.8.3 Proposta de acessibilidade com princípios de LIBRAS.....	46
3.9 Integração e Avaliação Final.....	46
3.9.1 Integração das soluções propostas.....	46
3.9.2 Avaliação dos resultados alcançados.....	47
3.9.3 Identificação de limitações técnicas e conceituais.....	48
3.9.4 Proposição de melhorias e evolução futura.....	49
3.9.5 Considerações finais sobre o projeto.....	50
4 CONCLUSÃO.....	51
5 REFERÊNCIAS.....	53

1 INTRODUÇÃO

O desenvolvimento de plataformas digitais voltadas ao ensino e à aprendizagem representa uma das áreas de maior crescimento no setor de tecnologia, tornando-se cada vez mais relevante para profissionais formados em Análise e Desenvolvimento de Sistemas. A capacidade de projetar sistemas que integrem avaliação, gestão de conteúdo e análise de desempenho exige o domínio articulado de diversas competências técnicas, o que torna esse tipo de solução um objeto de estudo especialmente adequado ao contexto do PIM III. Diante disso, este trabalho propõe o projeto de uma plataforma web de avaliação e apoio à aprendizagem para uma empresa fictícia de tecnologia educacional denominada Tech Quest, cujo problema central é a ausência de um ambiente digital unificado que permita a criação de cursos, a aplicação de avaliações e o acompanhamento do desempenho dos usuários de forma estruturada e acessível em diferentes dispositivos.

A empresa foi concebida para atender três perfis distintos de usuários: estudantes de cursos de tecnologia que buscam aprender programação de forma prática e flexível, tutores especialistas que desejam criar e disponibilizar conteúdos e avaliações, e administradores responsáveis pela moderação do sistema e pelo acompanhamento dos indicadores gerais da plataforma. O problema identificado que justifica a criação do sistema é a ausência de um ambiente digital unificado capaz de integrar criação de cursos, aplicação de avaliações, acompanhamento individualizado de desempenho e emissão de certificados de conclusão, de forma acessível em diferentes dispositivos e perfis de acesso.

O objetivo geral do trabalho é desenvolver o projeto completo da plataforma Tech Quest, integrando de forma prática os conhecimentos adquiridos nas disciplinas do semestre. Para isso, foram estabelecidos objetivos específicos que orientaram cada etapa do desenvolvimento: a definição do contexto organizacional da empresa fictícia, o levantamento e a priorização de requisitos com base em práticas ágeis, a modelagem do banco de dados relacional, a implementação de funcionalidades com programação orientada a objetos em C#, a construção de interfaces web responsivas, a elaboração de protótipos e fluxos de UX/UI design, a análise de indicadores de

desempenho dos usuários e a integração de todos os elementos em um sistema coerente e funcional. Aspectos de acessibilidade, comunicação em equipe e conformidade com a Lei Geral de Proteção de Dados (LGPD) também foram considerados ao longo do projeto.

A metodologia adotada baseou-se em princípios da engenharia de software ágil, com a organização das funcionalidades em backlog, definição de iterações e critérios de aceite. O protótipo foi desenvolvido pelo Stitch e o site com HTML, CSS e JavaScript seguindo arquitetura mobile-first e utilizando dados simulados em arquivos JSON para demonstrar as funcionalidades do sistema sem a necessidade de infraestrutura de back-end nesta fase do projeto. Foram definidas três personas representativas — Felipe Almeida (estudante), Ana Souza (tutora) e Carlos Mendes (administrador) — cujas necessidades e comportamentos orientaram as decisões de design e desenvolvimento em todas as etapas.

2 CONTEXTUALIZAÇÃO TEÓRICA

2.1 Engenharia de Software Ágil Aplicada

As metodologias ágeis surgiram como resposta às limitações dos modelos tradicionais de desenvolvimento de software, que frequentemente resultavam em projetos longos, inflexíveis e desalinhados às necessidades reais dos usuários. O Manifesto Ágil, publicado em 2001 por um grupo de dezessete desenvolvedores, estabeleceu os quatro valores e os doze princípios fundamentais dessa abordagem, priorizando indivíduos e interações em detrimento de processos e ferramentas, software funcionando em detrimento de documentação abrangente, colaboração com o cliente em detrimento de negociação contratual e resposta a mudanças em detrimento de seguir um plano rígido predefinido.

Entre os principais frameworks ágeis destacam-se o Scrum, o Kanban e o Extreme Programming (XP), cada um com práticas e cerimônias específicas voltadas à organização das equipes e à entrega contínua de valor. O Scrum organiza o trabalho em ciclos curtos chamados sprints, com duração típica de uma a quatro semanas, nos

quais a equipe planeja, executa e revisa um conjunto de funcionalidades priorizadas no backlog do produto. O Kanban, por sua vez, utiliza um quadro visual com colunas que representam os estágios do fluxo de trabalho, permitindo o acompanhamento em tempo real do progresso das tarefas e a identificação de gargalos no processo de desenvolvimento.

A adoção de práticas ágeis tem demonstrado resultados expressivos na indústria de software, com equipes relatando maior satisfação dos clientes, redução do tempo de entrega e melhor capacidade de adaptação a mudanças de requisitos. O levantamento e a priorização de requisitos, a definição de critérios de aceite e a organização iterativa do trabalho são práticas que contribuem diretamente para a qualidade do produto final, pois garantem que as funcionalidades desenvolvidas estejam sempre alinhadas às necessidades reais dos usuários e aos objetivos do negócio. Nesse sentido, a engenharia de software ágil representa não apenas um conjunto de técnicas, mas uma mudança de mentalidade na forma como equipes de tecnologia colaboram e entregam valor.

2.2 Modelagem de Banco de Dados e NoSQL

A modelagem de banco de dados é o processo de definição da estrutura lógica e física na qual as informações de um sistema serão organizadas, armazenadas e recuperadas. O modelo relacional, proposto por Edgar F. Codd em 1970, fundamenta-se na organização dos dados em tabelas compostas por linhas e colunas, com relacionamentos definidos por meio de chaves primárias e estrangeiras. Esse modelo é amplamente adotado em sistemas que exigem integridade referencial, consistência nas transações e suporte a consultas complexas, sendo implementado por sistemas gerenciadores como MySQL, PostgreSQL, Oracle e SQL Server.

Com o crescimento exponencial do volume de dados gerados por aplicações web e sistemas distribuídos, os bancos de dados NoSQL emergiram como alternativa ao modelo relacional para cenários que exigem alta escalabilidade, flexibilidade de esquema e desempenho em operações de leitura e escrita em larga escala. Os bancos NoSQL são classificados em quatro categorias principais: orientados a documentos, como o MongoDB; orientados a chave-valor, como o Redis; orientados

a colunas, como o Cassandra; e orientados a grafos, como o Neo4j. Cada categoria atende a casos de uso específicos, e a escolha entre SQL e NoSQL deve ser baseada nas características dos dados e nos requisitos de desempenho e consistência do sistema.

A modelagem conceitual, representada pelo Diagrama Entidade-Relacionamento (DER), é a primeira etapa do processo de modelagem e tem como objetivo representar as entidades do sistema, seus atributos e os relacionamentos entre elas de forma independente da tecnologia utilizada. A partir do modelo conceitual, elabora-se o modelo lógico, que detalha as tabelas, os tipos de dados, as chaves e as restrições de integridade, e posteriormente o modelo físico, que considera as particularidades do sistema gerenciador de banco de dados escolhido. Esse processo estruturado garante que o banco de dados seja projetado de forma consistente, eficiente e alinhada aos requisitos do sistema desde as etapas iniciais do desenvolvimento.

2.3 Programação Orientada a Objetos (C#)

A programação orientada a objetos é um paradigma de desenvolvimento de software que organiza o código em torno de objetos, que representam entidades com atributos — seus dados — e métodos — seus comportamentos. Proposto inicialmente por Alan Kay nos anos 1960 com a linguagem Smalltalk, o paradigma se consolidou como o modelo dominante no desenvolvimento de sistemas corporativos, sendo suportado por linguagens como Java, Python, C++ e C#. Os quatro pilares fundamentais da POO são o encapsulamento, que protege os dados internos de um objeto; a herança, que permite que uma classe reutilize atributos e comportamentos de outra; o polimorfismo, que permite que um mesmo método se comporte de maneira distinta conforme o tipo do objeto; e a abstração, que oculta a complexidade interna expondo apenas o que é relevante para o uso externo.

A linguagem C#, desenvolvida pela Microsoft e lançada em 2000 como parte da plataforma .NET, é uma das principais implementações do paradigma orientado a objetos na atualidade. Com tipagem estática forte, suporte a generics, delegates, eventos, LINQ e programação assíncrona com `async/await`, C# é amplamente

utilizada no desenvolvimento de aplicações web com ASP.NET, sistemas desktop com WPF, jogos com Unity e aplicações corporativas de grande porte. Sua sintaxe moderna, combinada com um ecossistema robusto de ferramentas e bibliotecas, torna a linguagem uma escolha consolidada tanto no mercado de trabalho quanto no ambiente acadêmico.

Os padrões de projeto, conhecidos como design patterns, representam soluções reutilizáveis para problemas recorrentes no desenvolvimento orientado a objetos, sendo documentados de forma sistemática pelo grupo Gang of Four na obra *Design Patterns: Elements of Reusable Object-Oriented Software*, publicada em 1994. Padrões como Singleton, Factory, Observer e Strategy são amplamente adotados em projetos C# para promover a flexibilidade, a manutenibilidade e a extensibilidade do código. A aplicação consciente desses padrões, combinada com princípios como os do SOLID — acrônimo que reúne cinco diretrizes fundamentais de design orientado a objetos — resulta em sistemas mais organizados, testáveis e preparados para evoluir com menor custo de manutenção ao longo do tempo.

2.4 Desenvolvimento Web Responsivo

O desenvolvimento web responsivo é uma abordagem de criação de interfaces digitais que se adaptam automaticamente a diferentes tamanhos de tela, resoluções e dispositivos, proporcionando uma experiência de uso consistente e adequada independentemente do meio de acesso. O conceito foi introduzido por Ethan Marcotte em 2010, em um artigo publicado na revista *A List Apart*, e rapidamente se tornou uma prática padrão na indústria de desenvolvimento web. Os fundamentos técnicos do design responsivo incluem o uso de grades fluidas, imagens flexíveis e media queries em CSS, que permitem que o layout reorganize seus elementos conforme as características do dispositivo do usuário.

Com o crescimento expressivo do acesso à internet por dispositivos móveis, que segundo o relatório *Global Digital da We Are Social* já supera o acesso por desktop em diversas regiões do mundo, a responsividade deixou de ser um diferencial e passou a ser um requisito essencial de qualidade para qualquer aplicação web. A abordagem *mobile-first*, que consiste em projetar o layout primeiramente para

dispositivos móveis e progressivamente adaptá-lo para telas maiores, tornou-se a estratégia mais recomendada por especialistas e empresas como o Google, que a adota como critério de ranqueamento em seus algoritmos de busca. Tecnologias como CSS Grid, Flexbox e unidades relativas de medida são os principais recursos utilizados para implementar layouts responsivos de forma eficiente e sem dependência de frameworks externos.

A semântica do HTML desempenha papel fundamental no desenvolvimento web responsivo, pois o uso correto de elementos como `<header>`, `<nav>`, `<main>`, `<section>`, `<article>` e `<footer>` contribui não apenas para a organização do código, mas também para a acessibilidade e a indexação do conteúdo por mecanismos de busca. A combinação de HTML semântico com CSS responsivo e JavaScript para interações dinâmicas forma a base do desenvolvimento web moderno, permitindo a criação de interfaces funcionais, acessíveis e adaptadas a qualquer contexto de uso. Esse conjunto de tecnologias, quando aplicado com boas práticas de desempenho e acessibilidade, resulta em aplicações web de alta qualidade que atendem a uma ampla diversidade de usuários e dispositivos.

2.5 UX e UI Design

A experiência do usuário, conhecida pela sigla UX do inglês User Experience, é uma disciplina que estuda e projeta a qualidade da interação entre pessoas e sistemas digitais, considerando aspectos como usabilidade, acessibilidade, eficiência, satisfação e emoção durante o uso. O conceito foi popularizado por Donald Norman na década de 1990, quando trabalhava na Apple, e desde então tornou-se uma área central no desenvolvimento de produtos digitais. O processo de UX design envolve pesquisa com usuários, criação de personas, mapeamento de jornadas, prototipagem e testes de usabilidade, com o objetivo de garantir que o produto final atenda de forma efetiva às necessidades reais de quem o utiliza.

O design de interface, conhecido como UI do inglês User Interface, complementa o UX ao tratar da composição visual dos elementos com os quais o usuário interage diretamente, como botões, formulários, tipografia, paleta de cores, ícones e espaçamento. Enquanto o UX se preocupa com o que o usuário sente e

consegue realizar, o UI se preocupa com como esses elementos são apresentados de forma clara, consistente e esteticamente agradável. A análise de usabilidade com base nas dez heurísticas propostas por Jakob Nielsen é uma das ferramentas mais utilizadas para avaliar a qualidade de uma interface, permitindo identificar problemas de interação e orientar melhorias antes mesmo da implementação definitiva do sistema.

A criação de personas é uma das práticas centrais do UX design, consistindo na elaboração de perfis semifictícios que representam os usuários reais de um sistema, com base em pesquisas e dados de comportamento. Cada persona é descrita com nome, idade, ocupação, objetivos, frustrações e padrões de uso, humanizando o processo de design e orientando as decisões da equipe de desenvolvimento em direção às necessidades concretas do público-alvo. Complementarmente, os wireframes e protótipos permitem visualizar e testar a estrutura das interfaces antes da implementação, reduzindo retrabalho e garantindo que a solução final seja validada com os usuários nas etapas iniciais do projeto.

2.6 Machine Learning e Análise de Dados

A análise de dados é o processo de inspeção, limpeza, transformação e modelagem de conjuntos de dados com o objetivo de descobrir informações úteis, identificar padrões e apoiar a tomada de decisão. Com o advento do conceito de Big Data e a capacidade crescente de armazenamento e processamento de grandes volumes de informação, a análise de dados tornou-se uma competência estratégica para organizações de todos os setores. Ferramentas como Python, com suas bibliotecas Pandas, NumPy e Matplotlib, e ambientes como o Jupyter Notebook são amplamente utilizados por cientistas e analistas de dados para explorar e visualizar informações de forma eficiente e reproduzível.

O machine learning, subcampo da inteligência artificial, desenvolve algoritmos capazes de aprender padrões a partir de dados históricos e realizar previsões ou classificações sobre novos dados sem serem explicitamente programados para isso. Os algoritmos de machine learning são classificados em três categorias principais: aprendizado supervisionado, no qual o modelo é treinado com dados rotulados;

aprendizado não supervisionado, no qual o modelo identifica padrões em dados sem rótulos; e aprendizado por reforço, no qual o modelo aprende por meio de tentativa, erro e recompensas. Cada abordagem é aplicável a diferentes tipos de problemas, desde a classificação de e-mails como spam até a recomendação personalizada de conteúdos em plataformas digitais.

No contexto educacional, a análise de dados e o machine learning têm sido aplicados para personalizar trilhas de aprendizado, prever o risco de evasão de estudantes e recomendar conteúdos com base no histórico de desempenho de cada usuário. Indicadores como taxa de aprovação, tempo médio de conclusão, índice de engajamento e padrões de acesso ao conteúdo fornecem informações valiosas para educadores e gestores pedagógicos, permitindo intervenções mais assertivas e personalizadas no processo de ensino e aprendizagem. Essa aplicação da ciência de dados à educação, denominada Learning Analytics, representa uma das fronteiras mais promissoras da tecnologia educacional contemporânea.

2.7 Comunicação, Liderança, Negociação

A comunicação é a base de qualquer processo colaborativo, sendo definida como a troca de informações, ideias e significados entre indivíduos por meio de canais verbais, não verbais ou escritos. No ambiente organizacional e nos projetos de desenvolvimento de software, a comunicação eficaz é determinante para o alinhamento entre membros da equipe, a clareza na definição de requisitos e a prevenção de conflitos. Estudos na área de gestão de projetos apontam que a maior parte das falhas em projetos de tecnologia está relacionada não a problemas técnicos, mas a falhas de comunicação entre as partes envolvidas, o que reforça a importância do desenvolvimento dessa competência na formação de profissionais de tecnologia da informação.

A liderança é a capacidade de influenciar, motivar e orientar pessoas em direção a objetivos comuns, sendo considerada uma das competências mais valorizadas no mercado de trabalho contemporâneo. A liderança situacional, conceito desenvolvido por Paul Hersey e Kenneth Blanchard, defende que o estilo de liderança mais eficaz varia conforme o nível de maturidade e competência da equipe, exigindo

do líder flexibilidade e sensibilidade para adaptar sua abordagem a cada contexto. Em ambientes ágeis, o papel do líder se transforma: em vez de comandar e controlar, o líder serve à equipe removendo impedimentos, facilitando a comunicação e criando as condições necessárias para que os membros trabalhem com autonomia e produtividade.

A negociação é um processo de comunicação orientado à construção de acordos mutuamente satisfatórios entre partes com interesses distintos ou conflitantes. No contexto de projetos de software, a negociação está presente em diversas situações, como a definição do escopo do produto, a priorização de funcionalidades no backlog, o estabelecimento de prazos e a resolução de conflitos entre membros da equipe ou com stakeholders externos. O método de negociação baseada em princípios, desenvolvido pelo Programa de Negociação da Universidade de Harvard, propõe que os negociadores se concentrem nos interesses subjacentes das partes, e não em posições rígidas, buscando soluções criativas que gerem valor para todos os envolvidos e preservem os relacionamentos a longo prazo.

2.8 LIBRAS

A Língua Brasileira de Sinais, conhecida pela sigla LIBRAS, é uma língua natural de modalidade visual-espacial utilizada pela comunidade surda brasileira. Reconhecida oficialmente como meio legal de comunicação pela Lei nº 10.436, de 24 de abril de 2002, e regulamentada pelo Decreto nº 5.626, de 22 de dezembro de 2005, LIBRAS possui gramática própria e independente da língua portuguesa, com estrutura linguística completa que contempla fonologia, morfologia, sintaxe e semântica expressos por meio de configurações de mão, movimentos, expressões faciais e corporais. Sua oficialização representou um marco histórico na conquista dos direitos linguísticos e educacionais da comunidade surda no Brasil.

A inclusão de LIBRAS nos currículos de cursos superiores, determinada pelo Decreto nº 5.626/2005, reflete o reconhecimento da importância dessa língua não apenas para surdos, mas para toda a sociedade, pois amplia as possibilidades de comunicação e promove a acessibilidade em diferentes contextos. No ambiente educacional, a presença de intérpretes de LIBRAS e a produção de materiais

acessíveis em língua de sinais são fundamentais para garantir a participação plena de estudantes surdos. Além disso, o conhecimento básico de LIBRAS por parte de profissionais de diversas áreas contribui para a quebra de barreiras comunicacionais e para a construção de uma sociedade mais inclusiva e equitativa.

No contexto do desenvolvimento de sistemas digitais, a acessibilidade para pessoas surdas vai além da simples tradução de conteúdos para LIBRAS, envolvendo o design de interfaces que considerem as especificidades perceptivas e cognitivas desse público. Recursos como legendas em vídeos, janelas de intérprete em língua de sinais, ícones visuais complementando textos e navegação que não dependa exclusivamente de conteúdo sonoro são exemplos de práticas de acessibilidade que beneficiam não apenas usuários surdos, mas toda a diversidade de perfis presentes em uma plataforma digital. O desenvolvimento de sistemas acessíveis em LIBRAS alinha-se às diretrizes da Web Content Accessibility Guidelines (WCAG) e representa um compromisso ético e social dos profissionais de tecnologia com a inclusão e com os direitos de todos os cidadãos.

3 DESENVOLVIMENTO

O grupo, tendo em vista o tema do PIM, desenvolveu a plataforma Tech Quest com o objetivo de criar um ambiente digital voltado ao apoio da aprendizagem e à aplicação de avaliações online, integrando conceitos estudados ao longo do semestre no curso. O projeto foi estruturado de forma interdisciplinar, envolvendo conhecimentos relacionados à engenharia de software ágil, modelagem de banco de dados, programação orientada a objetos em C#, desenvolvimento web responsivo e UX/UI design.

Inicialmente, foi realizada a definição do negócio fictício, identificando o segmento de atuação, o público-alvo e os principais problemas enfrentados pelos usuários. A empresa Tech Quest foi concebida para atuar no setor de tecnologia educacional, oferecendo uma plataforma capaz de integrar cursos, avaliações, acompanhamento de desempenho e emissão de certificados em um único ambiente digital.

Durante o levantamento de requisitos, foram identificadas funcionalidades essenciais para o funcionamento do sistema, como cadastro de usuários, criação de cursos, aplicação de avaliações, emissão de certificados e geração de indicadores de desempenho. Esses requisitos foram organizados em backlog e priorizados conforme as práticas da metodologia ágil Scrum, permitindo melhor organização das tarefas e definição das iterações de desenvolvimento.

Na etapa de modelagem de dados, foi elaborado um modelo relacional contendo tabelas, chaves primárias e relacionamentos necessários para garantir integridade e organização das informações do sistema.

A partir dos diagramas foi desenvolvido o protótipo através da tecnologia Stitch e as interfaces web foram realizadas utilizando HTML, CSS e JavaScript, seguindo a abordagem mobile-first para garantir responsividade em diferentes dispositivos. A utilização de CSS Grid, Flexbox e media queries possibilitou a adaptação do layout

para computadores, tablets e smartphones, proporcionando melhor experiência de uso aos usuários da plataforma.

Além dos aspectos técnicos, o projeto também considerou princípios de acessibilidade, usabilidade e conformidade com a Lei Geral de Proteção de Dados (LGPD). Foram definidos fluxos de navegação, personas e protótipos de interface, permitindo que as funcionalidades fossem desenvolvidas de maneira alinhada às necessidades reais dos usuários.

Por fim, a integração entre todas as etapas permitiu a construção de um protótipo funcional e coerente, demonstrando a aplicação prática dos conhecimentos adquiridos ao longo do curso e evidenciando a importância do planejamento, da organização e da interdisciplinaridade no desenvolvimento de sistemas.

3.1 Definição e Caracterização do Negócio Fictício

3.1.1 Denominação da empresa

A empresa fictícia criada para este projeto denomina-se Tech Quest, nome que remete à ideia de uma jornada de aprendizado em tecnologia. A identidade da empresa é construída em torno de valores como acessibilidade ao conhecimento, progressão contínua e engajamento dos usuários com o conteúdo educacional.

3.1.2 Segmento de atuação

A Tech Quest atua no segmento de tecnologia educacional, também conhecido como edtech. Seu foco está no desenvolvimento e na disponibilização de uma plataforma digital voltada ao ensino de programação e tecnologia da informação, integrando criação de conteúdo, avaliação e acompanhamento de desempenho em um único ambiente.

3.1.3 Público-alvo

A plataforma foi projetada para atender três perfis distintos de usuários:

— Estudantes: jovens e adultos interessados em aprender programação, com destaque para estudantes de cursos de tecnologia que buscam complementar sua formação de forma prática e flexível, como Felipe Almeida, 20 anos, cursando Análise e Desenvolvimento de Sistemas, que estuda no transporte público pelo celular.

— Tutores: profissionais e especialistas da área de tecnologia que desejam compartilhar conhecimento criando cursos e avaliações dentro da plataforma, como Ana Souza, 28 anos, engenheira de software sênior.

— Administradores: gestores responsáveis pelo funcionamento e pela moderação da plataforma, como Carlos Mendes, 35 anos, que acompanha os indicadores do sistema e aprova os conteúdos publicados pelos tutores.

3.1.4 Produtos e serviços oferecidos

A Tech Quest oferece os seguintes serviços por meio de sua plataforma web:

- Criação e publicação de cursos compostos por materiais de estudo e avaliações, realizada pelos tutores;
- Acesso a conteúdos educacionais em programação e tecnologia, disponíveis para os estudantes a qualquer momento e em qualquer dispositivo;
- Aplicação de provas e questionários com correção automatizada e registro de resultados;
- Acompanhamento do desempenho individual dos estudantes por meio de relatórios e indicadores;
- Sistema de gamificação com conquista de medalhas ao longo do progresso nos cursos;
- Emissão de certificados digitais de conclusão de curso;
- Painel administrativo para moderação de conteúdo e monitoramento geral da plataforma.

3.1.5 Necessidades e problemas que justificam a implantação do sistema

Antes da implantação da plataforma Tech Quest, a empresa identificou as seguintes necessidades e problemas no mercado de educação em tecnologia:

- Ausência de um ambiente digital unificado que integrasse criação de conteúdo, avaliação e acompanhamento de desempenho em um único sistema;
- Dificuldade dos estudantes em acompanhar seu próprio progresso de forma estruturada e motivadora;
- Falta de ferramentas acessíveis para que especialistas pudessem criar e disponibilizar conteúdos educacionais sem depender de grandes infraestruturas tecnológicas;
- Necessidade de interfaces responsivas e adaptadas ao uso em dispositivos móveis, considerando que grande parte do público estuda fora de ambientes fixos;
- Carência de relatórios e indicadores que auxiliem tutores e administradores na tomada de decisão pedagógica com base em dados reais de desempenho dos usuários.

3.2 Engenharia de Software Ágil Aplicada

3.2.1 Identificação das partes interessadas (stakeholders)

O desenvolvimento da plataforma Tech Quest envolveu a identificação das principais partes interessadas, cujas necessidades e expectativas orientaram o planejamento e a execução do projeto. Os stakeholders identificados foram:

- Estudantes: usuários finais da plataforma, interessados em acessar conteúdos de qualidade, realizar avaliações e acompanhar seu progresso de forma clara e motivadora;
- Tutores: profissionais responsáveis pela criação e publicação de cursos, interessados em ferramentas intuitivas para organizar materiais e avaliações;
- Administradores: gestores da plataforma, interessados em manter a qualidade do conteúdo publicado, monitorar indicadores e garantir o bom funcionamento do sistema;

— Equipe de desenvolvimento: responsável pela construção e manutenção da plataforma, interessada em requisitos claros, bem definidos e organizados.

3.2.2 Requisitos funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer para atender às necessidades dos usuários. Os principais requisitos funcionais levantados para a plataforma Tech Quest foram:

Tabela 1 - Requisitos Funcionais

ID	Requisito	Descrição
RF01	Gerir Utilizadores	Cadastrar, consultar, atualizar e desativar contas de usuários.
RF02	Autenticar Acesso	Validar a entrada de utilizadores via e-mail e senha com suporte a recuperação de credenciais.
RF03	Gerir Cursos	Criar, editar, arquivar e excluir cursos.
RF04	Moderar Publicações	Revisar, aprovar ou rejeitar cursos submetidos por tutores antes da publicação oficial.
RF05	Monitorizar Progresso	Registrar a visualização de materiais e calcular a percentagem de conclusão do curso.
RF06	Avaliar Conhecimento	Aplicar provas de múltipla escolha com correção automática e atribuição de nota.
RF07	Emitir Certificados	Gerar documento digital de conclusão com código de autenticidade para alunos aprovados.
RF08	Atribuir Medalhas	Conceder insígnias visuais ao perfil do estudante ao atingir marcos específicos (gamificação).
RF09	Protocolar Chamados	Abrir tickets de suporte técnico ou pedagógico direcionados aos responsáveis.
RF10	Interagir no Suporte	Responder, encaminhar e encerrar os chamados abertos pelos utilizadores.
RF11	Auditar Ações	Rastrear e armazenar logs de atividades críticas realizadas por administradores.
RF12	Extrair Relatórios	Gerar dashboards com o desempenho médio, taxa de evasão e estatísticas de conclusão.

Fonte: Os autores (2026).

3.2.3 Requisitos não funcionais

Os requisitos não funcionais definem as características de qualidade que o sistema deve apresentar, independentemente das funcionalidades específicas. Para a plataforma Tech Quest, foram definidos os seguintes requisitos não funcionais:

Tabela 2 - Requisitos Não Funcionais.

ID	Requisito	Descrição
RNF01	Segurança	Todas as senhas de utilizadores devem ser armazenadas no banco de dados utilizando criptografia (Hash).
RNF02	Integridade	O sistema não deve permitir a exclusão de cursos que possuam estudantes matriculados (deve-se usar o status "Arquivado").
RNF03	Usabilidade	A interface deve ser responsiva, permitindo que o estudante acesse os cursos tanto por computador como por dispositivos móveis.

Fonte: Os autores (2026).

3.2.4 Backlog do produto

O backlog do produto reúne todas as funcionalidades identificadas no levantamento de requisitos, organizadas por prioridade. A priorização foi realizada com base no valor entregue ao usuário final e na dependência entre as funcionalidades.

Tabela 3 - Backlog

ID	Atividade	Prioridade	Sprint	Situação
1	Definir público-alvo	ALTA	01	CONCLUÍDO
2	Definir serviços oferecidos	ALTA	01	CONCLUÍDO
3	Levantar requisitos funcionais	ALTA	02	CONCLUÍDO
4	Levantar requisitos não funcionais	ALTA	02	CONCLUÍDO
5	Criar backlog	ALTA	02	CONCLUÍDO
6	Criar Diagrama de Casos de Uso	ALTA	03	CONCLUÍDO
7	Criar Diagrama de Classes	ALTA	03	CONCLUÍDO
8	Criar Diagramas de Sequências	ALTA	03	CONCLUÍDO
9	Criar modelo conceitual (MER)	ALTA	04	CONCLUÍDO
10	Criar wireframes das telas	ALTA	05	CONCLUÍDO
11	Criar protótipo de alta fidelidade	ALTA	05	CONCLUÍDO
12	Desenvolver HTML e CSS	ALTA	06	CONCLUÍDO

Fonte: Os autores (2026).

3.2.5 Priorização das funcionalidades

A priorização das funcionalidades seguiu o método MoSCoW, amplamente utilizado em metodologias ágeis para classificar os itens do backlog conforme sua importância para o produto:

- Must have (deve ter): Gamificação, Responsividade e Segurança;
- Should have (deveria ter): Disponibilidade e Desempenho;
- Could have (poderia ter): Funcionalidades desenvolvidas de forma mais completas;
- Won't have (não terá nesta versão): Verificação de afinidade com o estudante, integração com back-end em C# e banco de dados em nuvem, previstas para o semestre seguinte.

3.2.6 Definição de iterações e critérios de aceite

O desenvolvimento da plataforma foi organizado em três iterações, cada uma com foco em um conjunto de funcionalidades prioritárias:

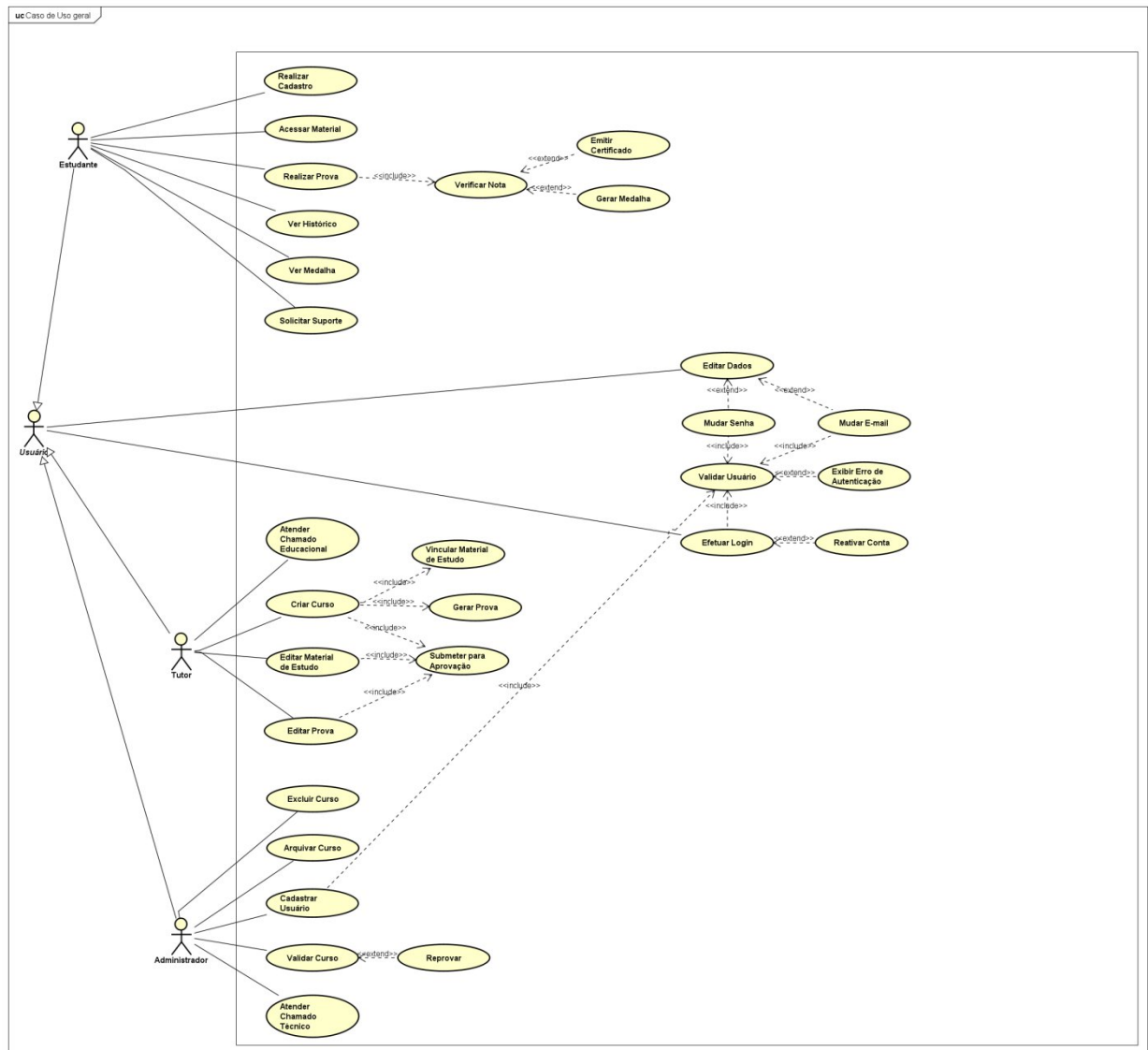
- Iteração 1 — Estrutura base: autenticação de usuários, navegação entre perfis e estrutura geral das páginas. Critério de aceite: todos os três perfis devem conseguir realizar login e acessar suas respectivas áreas sem erros.
- Iteração 2 — Funcionalidades principais: criação de cursos pelos tutores, inscrição e acesso pelos estudantes, realização de avaliações e registro de resultados. Critério de aceite: um estudante deve conseguir se inscrever em um curso, acessar o material, realizar a avaliação e visualizar sua nota ao final.
- Iteração 3 — Funcionalidades complementares: sistema de medalhas, emissão de certificados, painel administrativo e relatórios de desempenho. Critério de aceite: o estudante deve receber a medalha e o certificado ao concluir um curso com aprovação, e o administrador deve visualizar os indicadores atualizados no painel.

3.2.7 Diagramas

3.2.7.1 Diagrama de Caso de Uso

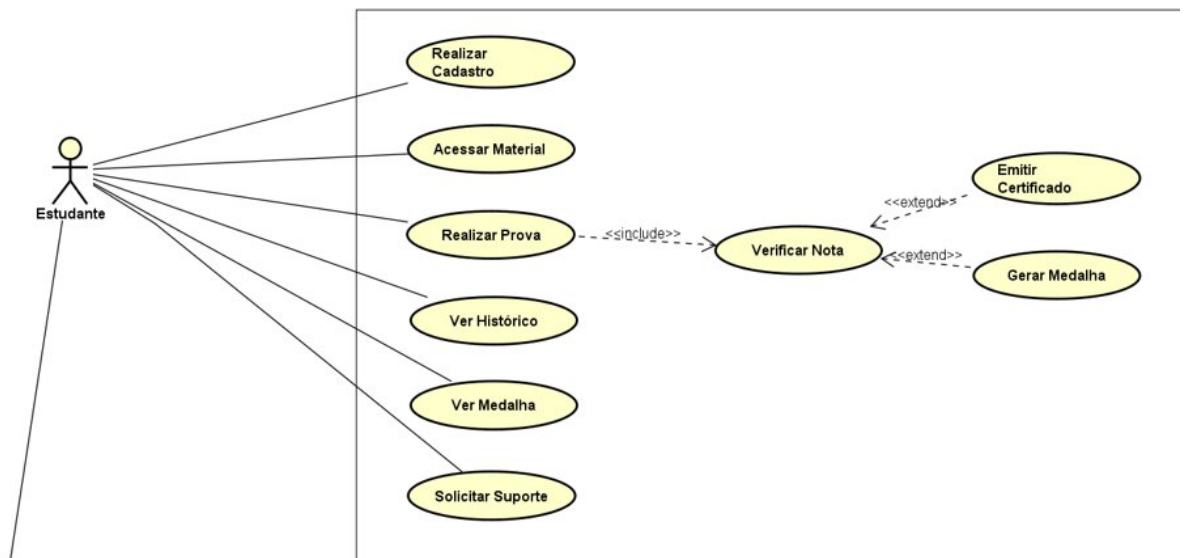
O Diagrama de Caso de Uso representa as interações entre os atores do sistema (Estudante, Tutor e Administrador) e as funcionalidades disponíveis na plataforma Tech Quest.

Figura 1 - Diagrama Casos de Uso



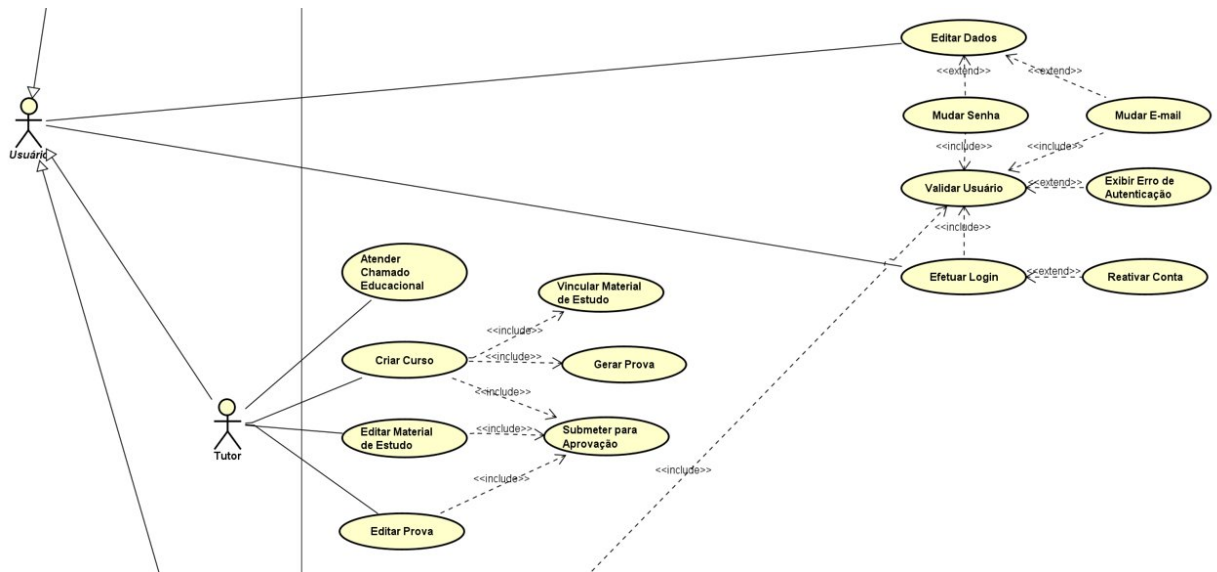
Fonte: Os autores (2026).

Figura 2 - Diagrama Caso de Uso Estudante



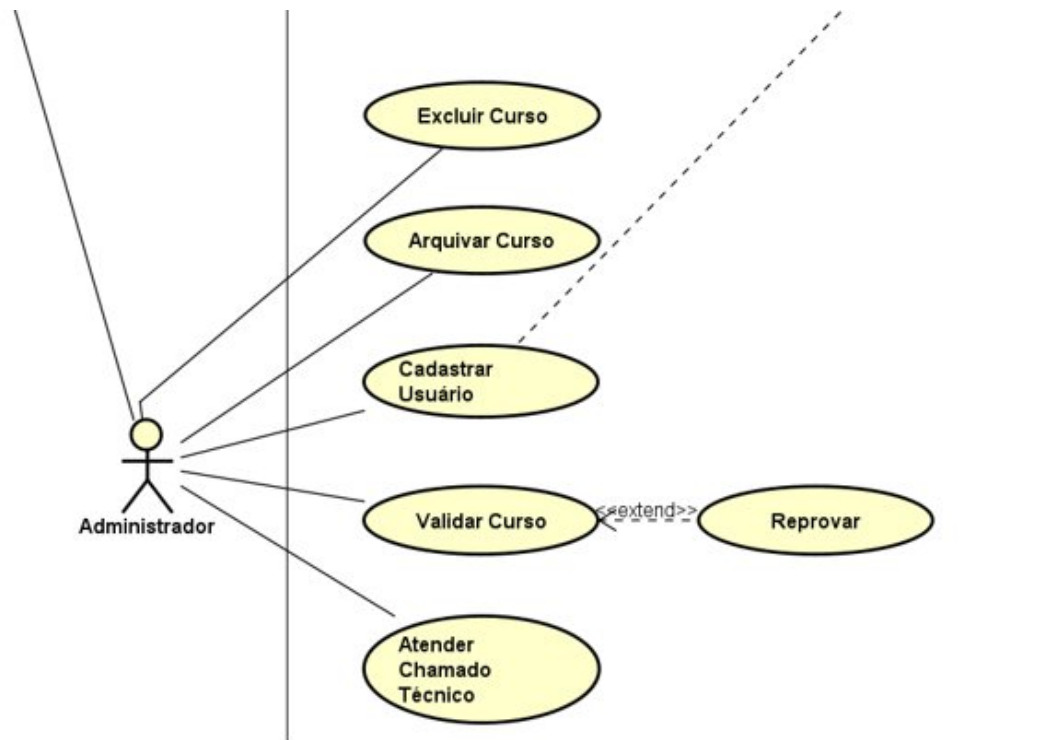
Fonte: Os autores (2026).

Figura 3 - Diagrama Caso De Uso Usuario



Fonte: Os autores (2026).

Figura 4 - Diagrama Caso de Uso Administrador



Fonte: Os autores (2026).

3.2.8 Sprints

Dividimos o trabalho em seis sprints, conforme o cronograma abaixo:

Tabela 4 - Sprints

Sprint	Período	Objetivo
01	02/03 – 09/03	Definir base do PIM
02	10/03 – 16/03	Definição de RF e RNF
03	17/03 – 16/04	Diagramas UML
04	17/04 – 24/04	Diagramas MER/DER
05	25/04 – 12/05	Protótipo Alta Fidelidade
06	13/05 – 22/05	Web site responsivo

Fonte: Os autores (2026).

- Na primeira sprint foi decidido o que iríamos realizar no projeto, definindo a base do PIM;
- Na segunda sprint foram levantados e documentados os Requisitos Funcionais e Não Funcionais do sistema;

- Na terceira sprint foram desenvolvidos os Diagramas UML do projeto;
- Na quarta sprint foram elaborados os Diagramas MER e DER para modelagem do banco de dados;
- Na quinta sprint foi criado o Protótipo de Alta Fidelidade do sistema;
- Na sexta sprint foi desenvolvido o Web Site responsivo com HTML , CSS e JavaScript.

3.3 Modelagem de Banco de Dados e NoSQL

3.3.1 Definição da Modelagem

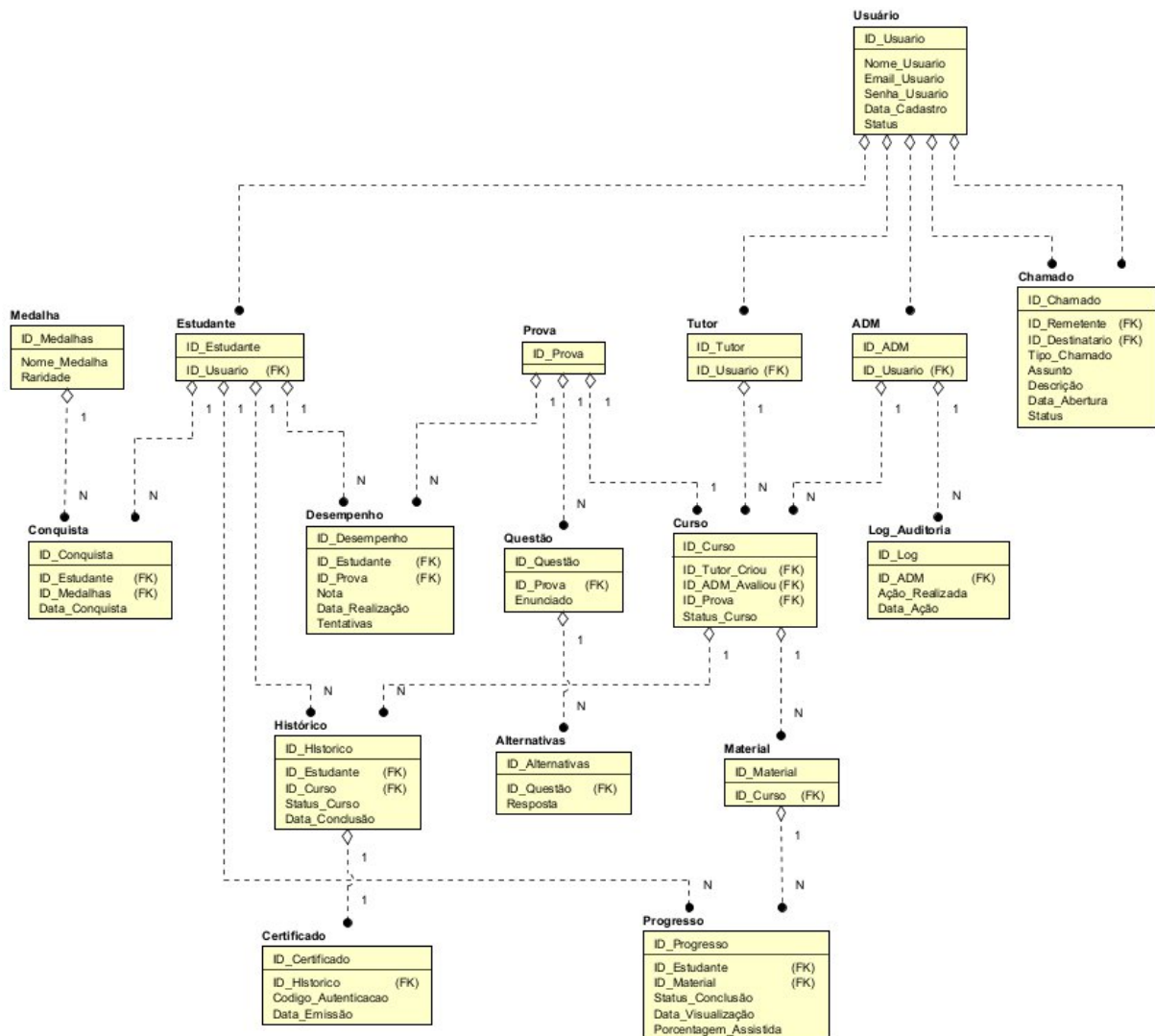
Durante a modelagem do sistema foram sendo definidas as entidades essenciais para o sistema e seus atributos, gerando o seguinte diagrama de entidade e relacionamento.

Para que a plataforma Tech Quest funcione adequadamente, é necessário armazenar e gerenciar um conjunto de informações relacionadas aos seus usuários, conteúdos e interações. As principais entidades identificadas a partir dos requisitos do sistema foram:

- Usuário: dados cadastrais e de autenticação de estudantes, tutores e administradores;
- Curso: informações sobre os cursos criados pelos tutores, incluindo título, descrição, categoria e status de aprovação;
- Material: conteúdos textuais ou referências de estudo vinculados a cada curso;
- Prova: avaliações associadas aos cursos, com suas respectivas questões e alternativas;
- Questão: cada questão individual que compõe uma Prova, com seu enunciado e alternativas;
- Alternativa: opções de resposta vinculadas a cada questão, com identificação da resposta correta;
- Resposta: registros das respostas submetidas pelos estudantes em cada prova realizada;

- Tentativa: registro de cada tentativa realizada pelo estudante ao realizar uma Prova;
- Nota: resultado obtido pelo estudante após a conclusão de cada tentativa;
- Desempenho: histórico consolidado de notas, progresso e pontuação acumulada por estudante;
- Medalha: conquistas desbloqueadas pelos estudantes com base em critérios de progresso, classificadas por Raridade (Bronze, Prata, Ouro ou Platina);
- Certificado: registros de conclusão de curso emitidos para estudantes aprovados;
- Chamado: registros de solicitações de suporte técnico ou educacional abertos por estudantes ou tutores;
- Sessão: controle de acessos autenticados ao sistema, com registro de data e hora;
- Histórico Acadêmico: consolidação da trajetória completa do estudante na plataforma;
- Inscrição: vínculo entre estudantes e cursos nos quais estão matriculados.

Figura 5 - Diagrama de Entidade e Relacionamento



Fonte: Os autores (2026).

3.3.2 Modelo conceitual de dados

O modelo conceitual representa as entidades do sistema e seus relacionamentos de forma independente da tecnologia de banco de dados utilizada. As principais entidades e relacionamentos identificados para a plataforma Tech Quest são:

- Usuário se cadastra com um perfil (estudante, tutor ou administrador);
- Tutor cria um ou mais Cursos;
- Administrador aprova ou rejeita Cursos submetidos pelos tutores;

- Estudante se inscreve em um ou mais Cursos;
- Curso contém um ou mais Materiais de estudo e uma ou mais Provas;
- Prova é composta por uma ou mais Questões, cada uma com suas respectivas Alternativas de resposta;
- Estudante realiza Provas por meio de Tentativas, tendo suas Respostas registradas a cada tentativa;
- Cada Tentativa concluída gera uma Nota associada ao desempenho do estudante;
- Estudante acumula Desempenho com base nos resultados das Provas realizadas;
- Estudante recebe Medalhas ao atingir critérios de progresso definidos, sendo cada Medalha classificada por uma Raridade (Bronze, Prata, Ouro ou Platina);
- Estudante recebe Certificado ao concluir um curso com aprovação;
- Estudante ou Tutor pode abrir um ou mais Chamados de suporte técnico ou educacional;
- Usuário inicia uma Sessão ao autenticar-se no sistema, com registro de data e hora de acesso;
- Estudante possui um Histórico Acadêmico que consolida sua trajetória completa na plataforma.

3.3.3 Justificativa técnica para a adoção de banco de dados relacional e/ou NoSQL

Banco de dados relacional (SQL)

A maior parte dos dados da plataforma Tech Quest possui estrutura bem definida, relacionamentos claros entre entidades e necessidade de integridade referencial, o que torna o banco de dados relacional a escolha mais adequada para o núcleo do sistema. Tabelas como usuários, cursos, avaliações e desempenho exigem consistência, transações seguras e consultas estruturadas, características fundamentais de sistemas gerenciadores de banco de dados relacionais como MySQL ou PostgreSQL.

Banco de dados não relacional (NoSQL)

O uso complementar de um banco de dados NoSQL é justificado para o armazenamento de dados com estrutura variável ou de alto volume de leitura e escrita, como os logs de acesso dos usuários e os registros de atividade na plataforma.

As operações e consultas mais relevantes para o funcionamento da plataforma Tech Quest são:

- Verificar se um estudante já está inscrito em determinado curso antes de permitir nova inscrição;
- Registrar as respostas de um estudante em uma Prova e calcular automaticamente a Nota obtida;
- Registrar o número de Tentativas realizadas por um estudante em cada Prova;
- Listar o histórico de desempenho de um estudante em todas as Provas realizadas;
- Verificar se um estudante atingiu a nota mínima para receber o Certificado de conclusão de um curso;
- Listar todos os cursos pendentes de aprovação para o painel do administrador;
- Consultar as Medalhas conquistadas por um estudante para exibição em seu perfil;
- Listar os Chamados abertos por perfil de usuário e direcionar ao responsável correspondente;
- Registrar e consultar as Sessões de acesso dos usuários para fins de auditoria e análise de engajamento.

3.4 Programação Orientada a Objetos (C#)

3.4.1 Arquitetura geral do projeto

Para a arquitetura do projeto, o grupo decidiu utilizar a plataforma Astah, resultando no diagrama de classes apresentado a seguir.

```

classDiagram
    class Chamado {
        -id: int
        -usuario: Usuario
        -assunto: string
        -descricao: string
        -categoria: string
        -status: string
        -prioridade: string
        -dataCriação: DateTime
        -anexos: List<File>
        +criar(): void
        +responder(mensagem: string): void
        +finalizar(): void
        +gerarProtocolo(): string
    }
    class Usuario {
        -id: int
        -nome: string
        -email: string
        -senha: string
        -dataCadastro: DateTime
        -contaAtivada: bool
        +autenticar(): bool
        +validarEmail(): bool
        +atualizarEmail(): bool
        +atualizarPerfil(): void
        +desativarConta(): void
    }
    class Sessao {
        -id: string
        -usuario: Usuario
        -token: string
        -dataInicio: DateTime
        -ativo: bool
        +criar(): void
        +validar(): bool
        +renovar(): void
        +encerrar(): void
    }
    class Estudante {
        -matricula: string
        -cursosMatriculados: List<Curso>
        -medidas: List<Medida>
        +matriculaCurso(): Curso
        +realizarProva(): Prova
        +visualizarMedidas(): List<Medidas>
    }
    class Tutor {
        -cursoGerenciado: List<Curso>
        -especialidadeMateria: string
        +criarCurso(): void
        +editarCurso(curso: Curso): void
        +gerarProva(curso: Curso): Prova
    }
    class Administrador {
        -nivelAcesso: string
        -permissões: List<String>
        +cadastroUsuario(): Usuario
        +validarCurso(curso: Curso): bool
        +atenderChamado(chamado: Chamado): Chamado
    }
    class Conquista {
        -id: int
        -estudante: Estudante
        -medalha: Medalha
        -dataConquista: DateTime
        -contexto: string
        +registrar(): void
        +notificar(): void
    }
    class Medalha {
        -nome: string
        -descricao: string
        -icone: string
        -nivelMedalha: int
        +exibir(): void
        +compartilhar(): void
    }
    class Nota {
        -id: int
        -prova(): Prova
        -estudante(): Estudante
        -valor(): double
        -dataRealizacao(): DateTime
        -respostas(): Map<int, String>
        -feedback(): string
        +calcular(): double
        +verificarAprovação(): bool
        +gerarCertificado(): bool
    }
    class Prova {
        -id: int
        -titulo(): string
        -descricao(): string
        -curso(): Curso
        -duraçãoMinutos(): int
        -dataDisponível(): DateTime
        -dataLimite(): DateTime
        +Iniciar(estudante: Estudante): void
        +Finalizar(): void
        +calcularNota(): double
        +adicionarQuestao(questao: Questao): void
    }
    class Questao {
        -id: int
        -enumerado: string
        -tipo: string
        -iposes: List<String>
        -pontuação: double
        +validarResposta(resposta: string): bool
    }
    class Curso {
        -id: int
        -nome: string
        -descricao: string
        -cargaHoraria: int
        -categoriaCurso: Categoria
        -tutor: Tutor
        -status: string
        -dataPublicação: DateTime
        +adicionarMaterial(material: Material): void
        +criarProva(): Prova
        +matricularEstudante(estudante: Estudante): void
        +equivale(): void
    }
    class Material {
        -id: int
        -titulo: int
        -conteudo: string
        -tipo: string
        -ordem: int
        -anexos: List<File>
        +visualizar(): void
        +editar(): void
        +registrarAcesso(estudante: Estudante): void
    }
    class Categoria {
        -id: int
        -nome: string
        -descricao: string
        +listarCursos(): List<Curso>
    }
    Chamado "1" -- "*" Usuario : - atre
    Chamado "1" -- "*" Chamado : - possui
    Chamado "1" -- "*" Chamado : - contaAtivada: bool
    Chamado "1" -- "*" Chamado : - atualizarPerfil(): void
    Chamado "1" -- "*" Chamado : - desativarConta(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - criar(): void
    Chamado "1" -- "*" Chamado : - responder(mensagem: string): void
    Chamado "1" -- "*" Chamado : - finalizar(): void
    Chamado "1" -- "*" Chamado : - gerarProtocolo(): string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
    Chamado "1" -- "*" Chamado : - dataCriação: DateTime
    Chamado "1" -- "*" Chamado : - anexos: List<File>
    Chamado "1" -- "*" Chamado : - descricao: string
    Chamado "1" -- "*" Chamado : - assunto: string
    Chamado "1" -- "*" Chamado : - id: int
    Chamado "1" -- "*" Chamado : - usuario: Usuario
    Chamado "1" -- "*" Chamado : - categoria: string
    Chamado "1" -- "*" Chamado : - status: string
    Chamado "1" -- "*" Chamado : - prioridade: string
   
```

O diagrama utiliza diversos conceitos de C#. As classes representam cada entidade do sistema com seus atributos e métodos. O encapsulamento é aplicado com membros privados (-) e públicos (+), protegendo os dados internos de cada classe. A herança aparece na relação entre Usuario e suas subclasses Estudante, Tutor e Administrador, que reutilizam os atributos e métodos comuns da classe base e especializam seus comportamentos com operações específicas de cada perfil. O polimorfismo permite que métodos como `autenticar()` sejam sobrescritos com comportamentos específicos em cada subclasse, enquanto operações como `atenderChamado()` existem exclusivamente no Administrador.

As composições entre Prova→Questão e Curso→Material representam objetos que contêm outros objetos diretamente em sua estrutura. As coleções genéricas (List<Curso>, List<Medalhas>, Map<int,String>) são amplamente usadas para representar relacionamentos de multiplicidade. Os tipos nativos do C# como string, int, bool, DateTime e File tipam os atributos das classes. Os métodos possuem assinaturas com parâmetros tipados e retornos definidos, como

`validarResposta(resposta: string): bool` e `matricularCurso(): Curso`, refletindo a tipagem estática da linguagem. As referências a objetos são vistas em atributos como `usuario: Usuario` dentro de `Sessão`, onde uma classe aponta para instância de outra. A classe `Conquista` registra o vínculo entre o `Estudante` e a `Medalha` obtida, armazenando a data e o contexto da conquista. Os cursos são organizados por meio da classe `Categoria`, que permite listar e filtrar os cursos disponíveis por área de conhecimento. A classe `Nota`, além de calcular e verificar a aprovação do estudante, é responsável por disparar a geração do `Certificado` por meio do método `gerarCertificado(): bool`, integrando diretamente o resultado da prova à emissão do documento de conclusão. A classificação das medalhas é controlada pelo atributo `nivelMedalha: int` da classe `Medalha`, que define o nível de dificuldade ou raridade de cada conquista disponível na plataforma.

3.5 Desenvolvimento Web Responsivo

3.5.1 Tecnologias utilizadas

O desenvolvimento das interfaces web da plataforma Tech Quest foi realizado utilizando exclusivamente tecnologias nativas da web, conforme orientação do projeto acadêmico. As tecnologias adotadas foram:

- HTML: utilizado para a estruturação semântica do conteúdo de todas as páginas da plataforma, com uso adequado de elementos como `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` e `<footer>`, garantindo acessibilidade e boa indexação do conteúdo;
- CSS: utilizado para a estilização visual das interfaces, definição do layout responsivo, animações e transições. Todo o estilo foi centralizado em um único arquivo externo `style.css`, vinculado a todas as páginas da aplicação;
- JSON: utilizado para simular os dados da aplicação, armazenando informações de usuários, cursos, avaliações, desempenho e medalhas em um arquivo `mock.json`, carregado dinamicamente pelo JavaScript via `fetch()`.
- JavaScript puro : utilizado para a implementação das interações dinâmicas da interface, como troca de perfil de usuário, carregamento de dados simulados e validações de formulários.

Nenhum framework, biblioteca externa ou sistema de build foi utilizado, garantindo que o projeto funcione diretamente ao abrir o arquivo index.html em um navegador, sem necessidade de servidor ou infraestrutura adicional.

3.5.2 Arquitetura do sistema

A plataforma Tech Quest foi desenvolvida seguindo a abordagem mobile-first, na qual o layout é projetado primeiramente para dispositivos móveis e, progressivamente, adaptado para telas maiores por meio de media queries. Essa decisão foi fundamentada no perfil da persona principal do sistema — Felipe Almeida, estudante que acessa a plataforma pelo smartphone no transporte público — tornando a experiência em dispositivos móveis a prioridade central do projeto.

3.5.3 Responsividade e adaptação de layout

A responsividade foi implementada exclusivamente com CSS puro, sem o uso de frameworks. O layout foi construído com CSS Grid e Flexbox, garantindo flexibilidade e adaptação natural a diferentes tamanhos de tela.

3.5.4 Principais telas desenvolvidas

As telas a seguir representam os principais pontos de interação de cada perfil de usuário com a plataforma Tech Quest:

Perfil Estudante:

- Dashboard: exibe os cursos em andamento, progresso geral, medalhas recentes e atalhos para as principais funcionalidades;
- Catálogo de cursos: lista todos os cursos aprovados disponíveis, com filtro por categoria e campo de busca;
- Detalhe do curso: apresenta a descrição completa, o conteúdo programático, o material de estudo e o botão de inscrição;
- Avaliação: exibe as questões de múltipla escolha uma a uma, com barra de progresso e temporizador;

- Resultado: apresenta a nota obtida, o gabarito comentado e, quando aprovado, o botão para emissão do certificado digital.

Perfil Tutor:

- Dashboard: resume os cursos criados, o número de estudantes inscritos e os indicadores de desempenho;
- Criar curso: formulário para cadastro de novo curso com título, descrição, categoria, material de estudo e questões da avaliação;
- Gerenciar cursos: lista os cursos do tutor com seus respectivos status e opções de edição.

Perfil Administrador:

- Dashboard: apresenta os principais indicadores da plataforma, como total de usuários, cursos ativos e avaliações realizadas;
- Aprovar cursos: lista os cursos pendentes de revisão com opções de aprovação ou rejeição;
- Indicadores gerais: exibe gráficos e tabelas com dados de desempenho agregado dos estudantes.

3.6 UX e UI Design

3.6.1 Definição de personas

As personas foram criadas com base nos três perfis de usuários identificados no levantamento de requisitos da plataforma Tech Quest:

Persona 1 — Felipe Almeida (Estudante)

Tabela 5 - Persona 1

Idade	20 anos
Ocupação	Estudante de Análise e Desenvolvimento de Sistemas, estagiário em suporte de TI
Dispositivo principal	Smartphone Android

Fonte: Os autores (2026).

Objetivos: Aprender programação de forma prática e progressiva; acompanhar seu desempenho e sentir que está evoluindo; estudar nos momentos livres, principalmente no transporte público.

Frustrações: Plataformas que não funcionam bem no celular; interfaces confusas que dificultam encontrar o conteúdo desejado; falta de feedback claro sobre seu progresso e desempenho.

Comportamento: Felipe acessa a plataforma em sessões curtas de 15 a 30 minutos. Prefere conteúdos objetivos e avaliações que possam ser realizadas sem interrupção. Valoriza conquistas visuais, como medalhas e barras de progresso, que reforçam sua motivação para continuar estudando.

Persona 2 — Ana Souza (Tutora)

Tabela 6- Persona 2

Idade	28 anos
Ocupação	Engenheira de software sênior em empresa de tecnologia
Dispositivo principal	Notebook

Fonte: Os autores (2026).

Objetivos: Compartilhar seu conhecimento técnico criando cursos de qualidade; acompanhar o desempenho dos estudantes inscritos em seus cursos; gerenciar seus conteúdos de forma simples e rápida.

Frustrações: Ferramentas com muitas etapas para publicar um conteúdo simples; falta de dados sobre como os estudantes estão respondendo ao seu material; interfaces pouco intuitivas que exigem tempo para aprender a usar.

Comportamento: Ana acessa a plataforma principalmente pelo notebook, em períodos noturnos após o trabalho. Tem familiaridade técnica elevada e valoriza eficiência, clareza nas informações e acesso rápido aos dados de desempenho dos seus alunos.

Persona 3 — Carlos Mendes (Administrador)

Tabela 7- Persona 3

Idade	35 anos
Ocupação	Gestor de plataformas digitais
Dispositivo principal	Desktop

Fonte: Os autores (2026).

Objetivos: Garantir a qualidade e a conformidade do conteúdo publicado na plataforma; monitorar os indicadores gerais de uso e desempenho do sistema; tomar decisões estratégicas com base em dados confiáveis.

Frustrações: Painéis administrativos sobrecarregados de informações sem hierarquia visual; falta de alertas e notificações sobre conteúdos pendentes de moderação; dificuldade em extrair relatórios claros e objetivos.

Comportamento: Carlos acessa a plataforma durante o horário comercial, pelo desktop. Prefere painéis com informações resumidas e bem organizadas, com acesso rápido às ações mais frequentes, como aprovação de cursos e visualização de indicadores.

3.6.2 Fluxos de navegação

Os fluxos de navegação descrevem os caminhos percorridos por cada persona dentro da plataforma Tech Quest para realizar suas principais tarefas. A definição dos fluxos orientou a organização das páginas e a hierarquia das informações em cada interface.

Fluxo do Estudante — Realizar uma avaliação:

Landing Page → Login → Dashboard do Estudante → Catálogo de Cursos → Detalhe do Curso → Material de Estudo → Avaliação → Resultado

3.6.3 Wireframes e protótipos

Os wireframes foram elaborados para representar a estrutura visual e a organização dos elementos em cada tela da plataforma antes da implementação. A seguir são descritas as decisões de layout tomadas nas principais telas de cada perfil.

Dashboard do Estudante:

A tela foi organizada em três blocos principais: um cabeçalho com saudação personalizada e indicadores rápidos (cursos em andamento, medalhas e pontuação), uma seção central com os cursos em progresso exibidos em formato de cards com barra de progresso individual, e uma seção lateral com as conquistas recentes e atalhos para as áreas mais acessadas. Em mobile, os blocos são empilhados verticalmente em coluna única, priorizando os cursos em andamento logo abaixo do cabeçalho.

Tela de Avaliação:

A tela de avaliação foi projetada para eliminar distrações e manter o foco do estudante. O layout apresenta uma questão por vez, com as alternativas exibidas como botões de seleção amplos e de fácil toque em dispositivos móveis. Uma barra de progresso no topo indica quantas questões já foram respondidas, e um contador exibe o número da questão atual em relação ao total. Os botões de navegação entre questões foram posicionados na parte inferior da tela, dentro da área de alcance natural do polegar em smartphones.

3.6.4 Sistema de design

Para garantir consistência visual em todas as telas da plataforma, foi definido um sistema de design com as seguintes diretrizes:

Paleta de cores:

Tabela 8 - Cores design

Elemento	Cor	Código Hex
Cor primária	Azul escuro	#1E3A5F
Cor secundária	Azul médio	#2E6DA4
Cor de destaque	Laranja	#F47C2B
Fundo principal	Branco	#FFFFFF
Fundo secundário	Cinza claro	#F4F6F8

Texto principal	Cinza escuro	#1A1A2E
Texto secundário	Cinza médio	#6B7280
Sucesso	Verde	#22C55E
Erro	Vermelho	#EF4444

Fonte: Os autores (2026).

3.6.5 Análise de usabilidade

A análise de usabilidade foi realizada com base nas dez heurísticas de Nielsen, identificando pontos de atenção e as melhorias aplicadas na interface da plataforma Tech Quest.

Tabela 9 - Heurísticas

Heurística	Situação identificada	Melhoria aplicada
Visibilidade do status do sistema	O estudante não sabia em que etapa da avaliação estava	Adicionada barra de progresso e contador de questões
Correspondência entre sistema e mundo real	Termos técnicos em inglês nos menus confundiam usuários iniciantes	Todos os rótulos traduzidos para português claro e objetivo
Controle e liberdade do usuário	Não havia como voltar à questão anterior durante a avaliação	Adicionado botão de navegação entre questões anteriores
Consistência e padrões	Botões de ação com estilos diferentes em páginas distintas	Padronização de todos os botões pelo sistema de design
Prevenção de erros	Formulário de criação de curso permitia submissão sem preencher todos os campos obrigatórios	Adicionadas validações em tempo real com mensagens de orientação
Reconhecimento em vez de memorização	Usuário precisava lembrar em qual curso estava inscrito para navegar	Adicionado histórico de navegação visível
Flexibilidade e eficiência de uso	Tutores experientes precisavam passar por muitas etapas para editar uma questão	Adicionado acesso direto à edição a partir da lista de cursos
Design estético e minimalista	Dashboard do administrador exibia informações em excesso na tela inicial	Reorganização com cards resumidos e expansão sob demanda

Fonte: Os autores (2026).

3.7 Machine Learning e Análise de Dados

3.7.1 Tipos de dados relevantes para análise

A plataforma Tech Quest gera e armazena diferentes tipos de dados ao longo da interação dos usuários com o sistema, constituindo a base para a geração de

indicadores de desempenho e apoio à tomada de decisão pedagógica. Os principais tipos identificados foram: dados de desempenho acadêmico (notas, tentativas e taxa de aprovação por curso); dados de engajamento (frequência de acesso e tempo gasto nos materiais); dados de progresso (percentual de conclusão e medalhas conquistadas); dados de perfil (informações cadastrais para segmentações comparativas); e dados de interação com o conteúdo (questões mais erradas e materiais com maior tempo de permanência).

3.7.2 Aplicação de técnicas de análise de dados

Foram aplicadas técnicas de análise descritiva e exploratória para identificar padrões de comportamento dos estudantes. A análise exploratória segmentou os estudantes em três grupos: alto desempenho (média acima de 8,0 e taxa de conclusão superior a 80%); desempenho intermediário (média entre 6,0 e 8,0, com lacunas em tópicos específicos); e baixo desempenho ou risco de evasão (média abaixo de 6,0 e baixa frequência de acesso nos últimos 14 dias). Essa segmentação permite que tutores e administradores identifiquem rapidamente quais estudantes necessitam de intervenção pedagógica.

3.7.3 Geração de relatórios e indicadores

Os indicadores foram organizados em três níveis de visualização conforme o perfil do usuário. Para o estudante: progresso geral, média geral, taxa de aprovação, medalhas conquistadas e cursos concluídos. Para o tutor: total de inscritos, média da turma, taxa de conclusão e questões com índice de erro superior a 60%. Para o administrador: total de usuários ativos, cursos publicados, avaliações realizadas, taxa de aprovação geral e crescimento mensal de inscrições.

3.7.4 Aplicação de machine learning

Foram propostas duas aplicações de machine learning. A primeira é um sistema de recomendação de cursos baseado em filtragem colaborativa, que sugere conteúdos ao estudante com base no comportamento de usuários com perfil semelhante. A segunda é um modelo preditivo de risco de evasão baseado em

regressão logística, que classifica cada estudante como baixo ou alto risco a partir de variáveis como dias sem acesso, percentual de conclusão dos cursos e média das últimas avaliações.

3.7.5 Interpretação dos resultados e recomendações

A análise simulada identificou três padrões principais. Primeiro, concentração de erros em questões específicas com índice superior a 70%, com recomendação de notificar o tutor para revisão do material correspondente. Segundo, queda de engajamento após a primeira semana de inscrição, com recomendação de implementar notificações automáticas de lembrete para estudantes inativos. Terceiro, correlação entre acesso exclusivo pelo smartphone e menor taxa de conclusão de avaliações, com recomendação de revisar a usabilidade das telas de prova na versão mobile.

3.8 Comunicação, Liderança, Negociação e LIBRAS

3.8.1 Estratégias de comunicação

O desenvolvimento da plataforma Tech Quest adotou estratégias de comunicação inspiradas nas práticas ágeis, garantindo alinhamento e transparência entre os membros da equipe. As principais práticas adotadas foram: reuniões semanais de alinhamento seguindo o modelo de sprint review e sprint planning do Scrum; documentação colaborativa com registro de decisões e alterações ao longo do projeto; comunicação assíncrona por mensagens instantâneas para esclarecimentos pontuais; e apresentações internas de progresso ao final de cada iteração. Essas estratégias contribuíram para a coesão da equipe e para a resolução ágil de conflitos.

3.8.2 Situações simuladas de liderança e negociação

Duas situações exigiram o exercício dessas competências. Na primeira, surgiram divergências sobre a priorização entre o sistema de medalhas e o módulo de relatórios de desempenho. A negociação foi conduzida pelo método de negociação

por princípios, resultando na priorização dos relatórios na segunda iteração e das medalhas na terceira, satisfazendo ambos os grupos sem comprometer o prazo. Na segunda, um membro enfrentou dificuldades técnicas no módulo de análise de dados. O líder adotou postura de liderança situacional, promovendo a redistribuição de tarefas e uma sessão de pair programming para transferência de conhecimento, sem comprometer a qualidade da entrega.

3.8.3 Proposta de acessibilidade com princípios de LIBRAS

A proposta visa garantir que estudantes surdos ou com deficiência auditiva utilizem a plataforma de forma plena e autônoma, organizada em três frentes. Na interface, estão previstas janela de intérprete em LIBRAS nos vídeos, legendas em todos os conteúdos audiovisuais conforme a NBR 15290, ícones visuais complementando os rótulos textuais e notificações visuais substituindo alertas sonoros. Na documentação, estão previstos um glossário bilíngue português-LIBRAS com termos técnicos, um vídeo de apresentação da plataforma em LIBRAS e descrições alternativas em todas as imagens e diagramas.

3.9 Integração e Avaliação Final

3.9.1 Integração das soluções propostas

A etapa de integração consolidou todas as soluções desenvolvidas ao longo das etapas anteriores em um sistema único, coerente e funcional. Cada componente do projeto foi revisado e ajustado para garantir que as decisões tomadas em uma etapa fossem consistentes com as demais, resultando em uma visão sistêmica e integrada da plataforma Tech Quest.

O quadro a seguir apresenta uma síntese das entregas realizadas em cada etapa e sua contribuição para o sistema final:

Tabela 10 - Contribuição para o sistema

Etapas	Entrega	Contribuição para o sistema
1	Definição do negócio fictício	Contexto organizacional e justificativa do sistema

2	Engenharia de software ágil	Backlog, requisitos e planejamento iterativo
3	Programação orientada a objetos	Lógica de negócio e regras do sistema em C#
4	Desenvolvimento web responsivo	Interfaces funcionais e adaptadas a diferentes dispositivos
5	UX e UI design	Experiência de uso, protótipos e sistema de design

Fonte: Os autores (2026).

A integração entre as camadas do sistema foi organizada conforme a arquitetura definida na etapa de programação orientada a objetos, com o front-end desenvolvido em HTML, CSS e JavaScript consumindo os dados simulados em JSON, a lógica de negócio implementada nas classes C# e a estrutura de dados sustentada pelo modelo relacional definido na etapa de modelagem. As personas definidas na etapa de UX orientaram tanto o desenvolvimento das interfaces quanto a definição dos indicadores de análise de dados, demonstrando a interdependência entre as diferentes áreas do projeto.

3.9.2 Avaliação dos resultados alcançados

A avaliação dos resultados foi realizada com base nos objetivos específicos definidos no início do projeto, verificando em que medida cada um foi alcançado ao longo do desenvolvimento.

Tabela 11- Resultado Objetivos

Objetivo específico	Resultado alcançado	Avaliação
Definir e caracterizar a empresa fictícia Tech Quest	Empresa definida com denominação, segmento, público-alvo, serviços e justificativa do sistema	Alcançado
Levantar e priorizar requisitos com práticas ágeis	Backlog definido com 12 funcionalidades priorizadas pelo método MoSCoW e organizadas em 3 iterações	Alcançado
Modelar os dados do sistema	Modelo conceitual e lógico elaborados com 10 tabelas, chaves e relacionamentos definidos	Alcançado
Desenvolver funcionalidades em C# com POO	Classes implementadas com encapsulamento, herança e polimorfismo aplicados	Alcançado
Construir interfaces web responsivas	16 páginas desenvolvidas em HTML, CSS e JavaScript com arquitetura mobile-first	Alcançado
Aplicar princípios de UX e UI design	Três personas definidas, fluxos de navegação mapeados, wireframes elaborados e análise heurística realizada	Alcançado

Gerar e analisar indicadores de desempenho	Indicadores definidos para os três perfis de usuário, modelo preditivo de evasão proposto	Alcançado
Considerar comunicação, liderança e LIBRAS	Estratégias de comunicação documentadas, situações de negociação simuladas e proposta de acessibilidade elaborada	Alcançado

Fonte: Os autores (2026).

3.9.3 Identificação de limitações técnicas e conceituais

O desenvolvimento da plataforma Tech Quest nesta fase apresentou limitações que devem ser reconhecidas de forma transparente, pois influenciam o escopo das entregas realizadas e orientam os próximos passos do projeto.

Limitações técnicas:

- Ausência de back-end real: o sistema opera exclusivamente com dados simulados em arquivos JSON, o que impede a persistência real de informações entre sessões, a autenticação segura de usuários e a execução de operações de escrita no banco de dados. Essa limitação é prevista para esta fase do projeto e será superada na próxima etapa, com a implementação do back-end em C# e a integração com banco de dados relacional;
- Ausência de banco de dados em produção: o modelo relacional foi projetado e documentado, mas não foi implementado em um sistema gerenciador de banco de dados real nesta fase, sendo os dados gerenciados por arquivos JSON estáticos;
- Algoritmos de machine learning não treinados com dados reais: os modelos preditivos propostos, como o de previsão de risco de evasão, foram documentados e exemplificados em código, mas não foram treinados e validados com conjuntos de dados reais de uso da plataforma, o que limita a avaliação de sua efetividade prática;
- Testes automatizados não implementados: por limitações de tempo e escopo desta fase, não foram desenvolvidos testes unitários ou de integração para as classes C# implementadas, o que representa uma lacuna a ser suprida nas próximas iterações do projeto.

Limitações conceituais:

- Empresa e usuários fictícios: por se tratar de um projeto acadêmico, todas as personas, dados de desempenho e cenários de uso foram simulados, o que limita a validade externa dos padrões identificados na análise de dados e das conclusões obtidas a partir deles;
- Validação com usuários reais não realizada: os wireframes e protótipos foram avaliados com base nas heurísticas de Nielsen, mas não foram submetidos a testes de usabilidade com usuários reais, o que poderia revelar problemas de interação não identificados pela análise heurística;
- Escopo reduzido em relação ao sistema completo: algumas funcionalidades identificadas no levantamento de requisitos, como integração com sistemas externos, suporte a videoaulas e sistema de pagamentos, foram classificadas como Won't have nesta fase e não foram desenvolvidas, representando lacunas em relação ao escopo completo de uma plataforma educacional real.

3.9.4 Proposição de melhorias e evolução futura

Com base nos resultados alcançados e nas limitações identificadas, foram propostas as seguintes melhorias para a plataforma Tech Quest. As prioridades imediatas são a implementação do back-end em C# com ASP.NET Core, expondo uma API RESTful para comunicação com o front-end, e a integração com banco de dados relacional MySQL, substituindo os arquivos JSON por operações reais de leitura e escrita. Estão previstas ainda a autenticação segura com JWT, o desenvolvimento de testes unitários para as classes de negócio e a hospedagem da aplicação em ambiente de nuvem como Microsoft Azure ou AWS.

Em etapas subsequentes, o projeto prevê o treinamento dos modelos de machine learning com dados reais, a implementação do sistema de recomendação de cursos por filtragem colaborativa, o desenvolvimento de aplicativo móvel nativo para iOS e Android e a expansão do sistema de gamificação com novas medalhas e trilhas de aprendizado personalizadas. A proposta de acessibilidade em LIBRAS será implementada integralmente, com produção dos vídeos em língua de sinais e do glossário bilíngue. Por fim, estão previstas a expansão do catálogo para outras áreas como design e segurança da informação, a implementação de um módulo de

aprendizado adaptativo que ajuste o nível de dificuldade com base no desempenho de cada estudante, a internacionalização da plataforma com suporte a múltiplos idiomas e a obtenção de certificação de acessibilidade digital.

3.9.5 Considerações finais sobre o projeto

O desenvolvimento da plataforma Tech Quest ao longo do PIM III proporcionou à equipe uma experiência prática abrangente, integrando de forma efetiva os conhecimentos adquiridos nas disciplinas do semestre em um projeto único e coerente. A necessidade de tomar decisões técnicas e de design de forma interdependente — considerando simultaneamente os requisitos do sistema, a estrutura do banco de dados, a lógica de negócio em C#, a responsividade das interfaces e a experiência do usuário — evidenciou a complexidade real do desenvolvimento de software e a importância da visão sistêmica na formação do tecnólogo em Análise e Desenvolvimento de Sistemas.

A abordagem ágil adotada ao longo do projeto demonstrou sua efetividade na organização do trabalho em equipe, permitindo adaptações de escopo e redistribuição de tarefas sem comprometer a qualidade das entregas.

4 CONCLUSÃO

O presente trabalho alcançou seu objetivo geral ao desenvolver o projeto completo da plataforma web Tech Quest, integrando de forma prática e interdependente os conhecimentos das disciplinas do semestre. Ao longo do desenvolvimento do projeto, foi possível percorrer o ciclo completo de concepção de um sistema de software, desde a definição do contexto organizacional da empresa fictícia até a avaliação crítica dos resultados obtidos, passando pelo planejamento ágil, a modelagem de dados, a implementação orientada a objetos, o desenvolvimento das interfaces, o design de experiência do usuário, a análise de dados e a gestão do trabalho em equipe.

Os objetivos específicos foram alcançados em sua totalidade, com exceção parcial da validação dos modelos de machine learning, cuja efetividade plena depende de dados reais de uso da plataforma, indisponíveis nesta fase do projeto. A empresa fictícia Tech Quest foi definida com clareza, os requisitos foram levantados e priorizados com base em práticas ágeis, o banco de dados foi modelado com consistência, as funcionalidades principais foram implementadas em C# com aplicação dos pilares da programação orientada a objetos, as interfaces foram construídas com responsividade e atenção à experiência do usuário, e os indicadores de desempenho foram definidos e documentados para apoio à tomada de decisão pedagógica.

O desenvolvimento do projeto evidenciou que a construção de um sistema de software de qualidade exige muito mais do que domínio técnico isolado. A necessidade de alinhar decisões entre diferentes camadas do sistema — garantindo que os requisitos levantados na etapa de engenharia ágil se refletissem na modelagem de dados, que a modelagem sustentasse a lógica implementada em C#, e que a lógica se traduzisse em interfaces coerentes com as personas definidas no UX design — demonstrou na prática a importância da visão sistêmica e da integração entre as áreas de conhecimento do curso de Análise e Desenvolvimento de Sistemas.

As limitações identificadas ao longo do projeto, como a ausência de back-end real, a operação com dados simulados em JSON e a não realização de testes de usabilidade com usuários reais, não representam falhas do processo, mas sim o reconhecimento honesto dos limites do escopo desta fase e a base para o planejamento das próximas etapas. A transparência na identificação dessas limitações é, ela mesma, um resultado do amadurecimento técnico e analítico da equipe ao longo do semestre, refletindo uma postura profissional de responsabilidade sobre as próprias entregas.

Por fim, o desenvolvimento da plataforma Tech Quest confirmou que os profissionais de tecnologia precisam ser capazes não apenas de escrever código, mas de compreender negócios, comunicar ideias, liderar equipes, projetar experiências inclusivas e tomar decisões fundamentadas em dados. O PIM III cumpriu seu papel ao proporcionar um espaço de aprendizado onde essas competências puderam ser exercitadas de forma integrada, colaborativa e orientada à resolução de um problema real, ainda que em contexto fictício, preparando os integrantes da equipe para os desafios concretos da vida profissional em tecnologia.

5 REFERÊNCIAS

- BECK, Kent et al. Manifesto para o desenvolvimento ágil de software. 2001. Disponível em: <https://agilemanifesto.org/iso/ptbr/manifesto.html>. Acesso em: 19 mai. 2026.
- BRASIL. Lei nº 10.436, de 24 de abril de 2002. Dispõe sobre a Língua Brasileira de Sinais – LIBRAS. Diário Oficial da União, Brasília, DF, 25 abr. 2002.
- BRASIL. Decreto nº 5.626, de 22 de dezembro de 2005. Regulamenta a Lei nº 10.436/2002. Diário Oficial da União, Brasília, DF, 23 dez. 2005.
- CODD, Edgar F. A relational model of data for large shared data banks. Communications of the ACM, New York, v. 13, n. 6, p. 377-387, jun. 1970.
- DATE, C. J. Introdução a sistemas de bancos de dados. 8. ed. Rio de Janeiro: Elsevier, 2004.
- FISHER, Roger; URY, William; PATTON, Bruce. Como chegar ao sim: como negociar acordos sem fazer concessões. 3. ed. Rio de Janeiro: Solomon, 2014.
- FOWLER, Martin. UML essencial: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.
- GAMMA, Erich et al. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.
- HERSEY, Paul; BLANCHARD, Kenneth. Psicologia para administradores: a teoria e as técnicas da liderança situacional. São Paulo: EPU, 1986.
- MARCOTTE, Ethan. Responsive web design. A List Apart, 2010. Disponível em: <https://alistapart.com/article/responsive-web-design>. Acesso em: 19 mai. 2026.
- MICROSOFT. Documentação do C#. Disponível em: <https://learn.microsoft.com/pt-br/dotnet/csharp>. Acesso em: 20 mai. 2026.
- NIELSEN, Jakob. 10 usability heuristics for user interface design. Nielsen Norman Group, 1994. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics>. Acesso em: 19 mai. 2026.
- NORMAN, Donald. The design of everyday things. New York: Basic Books, 2013.
- PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: McGraw-Hill, 2016.
- PROVOST, Foster; FAWCETT, Tom. Data science para negócios. Rio de Janeiro: Alta Books, 2016.
- QUADROS, Ronice Müller de; KARNOPP, Lodenir Becker. Língua de sinais brasileira: estudos linguísticos. Porto Alegre: Artmed, 2004.

RUSSELL, Stuart; NORVIG, Peter. Inteligência artificial: uma abordagem moderna. 4. ed. Rio de Janeiro: LTC, 2022.

SCHWABER, Ken; SUTHERLAND, Jeff. O guia do Scrum. 2020. Disponível em: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-PortugueseBR.pdf>. Acesso em: 20 mai. 2026.

W3C. Web Content Accessibility Guidelines (WCAG) 2.1. 2018. Disponível em: <https://www.w3.org/TR/WCAG21>. Acesso em: 20 mai. 2026.

WE ARE SOCIAL. Digital 2024: global overview report. Disponível em: <https://wearesocial.com/global-digital-report>. Acesso em: 20 mai. 2026.